



Overview

This document explains the current **Requirements Export** functionality end-to-end. It covers the user experience, data and component flow, export scoping rules, template system, parameter/glossary integration, and the behavior of each supported export format.

The feature is primarily implemented in:

- `frontend/src/pages/Requirements/RequirementsPage.tsx`
 - `frontend/src/components/requirements/ExportBuilder.tsx`
 - Supporting utilities/services:
 - `frontend/src/utls/requirementExportTemplates.ts`
 - `frontend/src/utls/exportDocx.ts`
 - `frontend/src/utls/exportPdfLayout.ts`
 - `frontend/src/services/parameter.service.ts`
 - `frontend/src/services/definitionEntry.service.ts`
-

High-Level User Flows

1. Opening the Export UI from the toolbar

1. User opens the Requirements page for a project.
2. In the top toolbar, the user clicks the **Data** dropdown.
3. Inside that dropdown, the user clicks the **Export** button.
4. This click:
 - Sets `isExportOpen` to `true`.
 - Closes the Data dropdown (`dataDropdownOpen` set to `false`).
5. Because `isExportOpen` is now `true`, the page renders the `ExportBuilder` component near the bottom of `RequirementsPage`.
6. Since this path does **not** set an `exportScope`, the export starts with **all requirements in the project** as the initial dataset (subject to how `allRequirements` is loaded).

Visually, this is the “global” export entry point: the export modal opens without pre-filtering to a specific component, function, test plan, or test case.

2. Opening the Export UI from scoped context (context menus)

Elsewhere in the UI (e.g. requirement context menus, verification/test-plan context), the export feature can be opened with a pre-defined **scope**, by setting:

- `exportScope = { type, id, label }`
- `isExportOpen = true`

When `exportScope` is set, the export modal still opens in the same way, but the initial `requirements` that are passed into `ExportBuilder` are **filtered** to match that scope:

- `type: 'component'` → requirements with `componentId === exportScope.id`.
- `type: 'function'` → requirements allocated to that function via `allocationLinks` with:
 - `sourceType === 'requirement'`
 - `targetType === 'function'`
 - `linkType === 'allocated_to'`
- `type: 'test_case'` → requirements linked to that test case via `requirementTestCaseLinks`.
- `type: 'test_plan'` → requirements that have at least one linked test case belonging to the plan's `planCases`.

For these context-based entries, the export modal also receives a `scopeLabel` and a `scopeFilenameSuffix` so that the generated file name can reflect the scope (e.g. `component_FooController`).

Data Flow & Architecture

Components and state at a glance

- `RequirementsPage`
 - Owns list/pagination/filtering state for requirements.
 - Fetches:
 - Paginated requirements (`paginatedData`) via `requirementService.getRequirements`.
 - Full, non-paginated requirements (`allRequirementsLive`) via `requirementService.getAllRequirements`.

- Baseline information (`baseline`) when a `baselineId` is present.
- Manages UI state for:
 - Export (`isExportOpen` , `exportScope`).
 - Import, baselines, traceability matrix, diagrams, quality panel, etc.
- When `isExportOpen` is true, it selects the appropriate set of requirements and passes them, along with additional context data, into `ExportBuilder` .
- **ExportBuilder**
 - A modal-like component responsible for **configuring** and **executing** the export.
 - Receives:
 - The requirements to export.
 - Project information (ID/name).
 - Optional scope label and filename suffix.
 - Component/function trees and linkage information.
 - Internally manages:
 - Export format selection.
 - Column selection.
 - Scope selection (when allowed).
 - Parameter resolution and glossary/abbreviation options.
 - Template selection and editing.
 - Step-based UI (format → scope → options → review).
 - Invokes underlying utilities (XLSX, jsPDF, docx, ReqIF builder) to actually generate and trigger the browser download.

Baseline vs live requirements

- When viewing a **live** project (no `baselineId` search param), `RequirementsPage` uses:
 - `requirementService.getRequirements(projectId, serverFilters)` for the paged table/document view.
 - `requirementService.getAllRequirements(projectId)` for the full list used by export and some other tools.
 - When viewing a **baseline** (`baselineId` present in the URL), the page loads baseline-specific data and behaves differently for some controls (e.g. import is disabled). For export, the implementation attaches `requirements` and `allRequirements` appropriate to the current view, and those are what `ExportBuilder` will see.
-

RequirementsPage Integration Details

Toolbar and Export entry point

In `RequirementsPage`, the top-right toolbar includes a **Data** dropdown which groups data-management actions:

- Import
- Export
- Baselines

The **Export** button inside that dropdown:

- Calls `setIsExportOpen(true)`.
- Closes the dropdown (`setDataDropdownOpen(false)`).
- Does **not** set an `exportScope` (this is important: the resulting export is initially “global” rather than scoped).

Computing the export dataset

When `isExportOpen && projectId`, `RequirementsPage` renders `ExportBuilder` inside an inline IIFE. It first computes:

- A boolean `isFromContextMenu = !!exportScope`.
- A set of `exportRequirements` according to:
 - If `exportScope` is falsy:
 - `exportRequirements = allRequirements`.
 - If `exportScope` is truthy:
 - For `type === 'component'`:
 - Filter by `componentId === exportScope.id`.
 - For `type === 'function'`:
 - Filter to requirements that are allocated to `exportScope.id` via `allocationLinks ( requirement → function , linkType === 'allocated_to' )`.
 - For `type === 'test_case'`:
 - Filter to requirements that have a `requirementTestCaseLink` with `targetId === exportScope.id`.
 - For `type === 'test_plan'`:

- Look up the plan in `verificationPlansList` .
- Collect case IDs from `plan.planCases` .
- Filter to requirements that have at least one test-case link whose target is in that case ID set.
- For any other/unexpected types, fall back to all requirements.

It also computes a `scopeFilenameSuffix` when `exportScope` is present by:

- Taking `exportScope.label` .
- Stripping any prefix before a colon (`:`).
- Replacing whitespace with underscores.
- Removing non-alphanumeric/underscore/hyphen characters.
- Truncating to at most 40 characters.

This suffix is used to construct a more informative filename for the exported file.

Props passed into `ExportBuilder`

The `ExportBuilder` instance is created with:

- `requirements` : `exportRequirements` as computed above.
- `projectName` : currently the `projectId` string.
- `projectId` : the current project's ID.
- `onClose` : a function that:
 - Sets `isExportOpen` to `false` .
 - Clears `exportScope` .
- `scopeLabel` : `exportScope?.label` (for display).
- `scopeFilenameSuffix` : the sanitized suffix described above.
- `enableScopeSelection` :
 - `false` when the export was opened from context (`isFromContextMenu` is true).
 - `true` when opened from the top toolbar (global export).
- `componentTree` : populated when `!isFromContextMenu` so the modal can offer scope selection by component.
- `functions` : populated similarly for function-based scoping.
- `allocationLinks` : provided (for non-context-menu exports) so the modal can derive coverage information and, when appropriate, scope by function.
- `requirementTestCaseLinks` : always passed so the modal can:

- Compute coverage (requirements ↔ test cases).
- Filter by test case/plan where applicable.

From a data perspective, `RequirementsPage` is responsible for choosing **which requirements and auxiliary data** the export builder sees, while `ExportBuilder` is responsible for **how** to turn that data into files.

ExportBuilder Internals

`ExportBuilder` is a multi-step wizard-like component with several responsibilities:

- Allowing the user to choose:
 - Export format.
 - Scope (when allowed).
 - Included columns.
 - Parameter resolution mode.
 - Glossary/abbreviation inclusion.
 - Layout/templates for PDF/Word.
- Applying and managing named export templates, per project.
- Integrating parameters and definition entries into the exported document.
- Building the final artifact and triggering the file download.

Core props

- `requirements: Requirement[]`
 - The initial set of requirements to consider for export.
 - May be scoped already (if opened from a context menu).
- `projectName?: string`
 - Used for display and in document cover pages (PDF/Word) when available.
- `projectId: string`
 - Used for fetching parameters, definition entries, and project-specific templates.
- `onClose: () => void`
 - Called on Escape key or when the user closes the modal.
- `scopeLabel?: string`
 - Human-readable descriptor for the scoped export (e.g. `"Component: F00-123"`).

- `scopeFilenameSuffix?: string`
 - Sanitized suffix used to distinguish output filenames for scoped exports.
- `enableScopeSelection?: boolean`
 - If `true`, the user can choose to narrow the export to:
 - All requirements.
 - A specific component.
 - A specific function.
 - If `false`, the initial scope is fixed to the passed-in `requirements`.
- `componentTree?: ComponentTreeNode[]`
 - Hierarchical PBS component tree used to present component scope options.
- `functions?: SystemFunction[]`
 - Function tree used for function scope options (flattened to a path-based label).
- `allocationLinks?: Link[]`
 - Requirement-to-function allocations used for:
 - Coverage computation.
 - Function-based scoping.
- `requirementTestCaseLinks?: RequirementTestCaseLinkLike[]`
 - Links from requirements to test cases.
 - Used for coverage info and test-based export scoping in cooperation with `RequirementsPage`.

Important internal state

- **Format and columns**
 - `selectedFormat: 'csv' | 'excel' | 'pdf' | 'word' | 'reqif'`
 - `columns: ExportColumn[]`
 - Starts from a `defaultColumns` list (ID, title, description, priority, status, category, owner, etc.).
 - Can be overridden/merged by applying a saved template (`mergeColumnsWithDefaults`).
 - `includeHeader: boolean`
 - Whether to include a header row in CSV/Excel, and it can inform PDF/Word table headers.
- **Scope and filtering**
 - `scopeType: 'all' | 'component' | 'function'`
 - `selectedComponentId: string`

- `selectedFunctionId`: string
- `componentSearch`: string
- `functionSearch`: string
- `flatComponents` and `flatFunctions`:
 - Derived via helper functions that flatten the component and function trees into linear lists with "A > B > C" display labels.
- `effectiveRequirements`:
 - When `enableScopeSelection` is `false`, this is exactly the incoming requirements.
 - When `enableScopeSelection` is `true`:
 - `scopeType === 'all'` → all requirements.
 - `scopeType === 'component'` → requirements whose `componentId` matches `selectedComponentId`.
 - `scopeType === 'function'` → requirements that are allocated to `selectedFunctionId` per `allocationLinks`.

• Coverage / linkage

- `allocationByReqId`: Map<requirementId, Link[]>
- `testsByReqId`: Map<requirementId, RequirementTestCaseLinkLike[]>
- Helper `getCoverageInfo(reqId)` returns:
 - `hasAllocation`: boolean
 - `hasVerification`: boolean
 - `coverageStatus`: 'OK' | 'Missing Tests' | 'Missing Allocation' | 'Missing Both'
- This coverage information can be used to:
 - Enrich the export (e.g. derived coverage columns or sections).
 - Drive later UI warnings or derived metrics in templates.

• Scope label and filename suffix

- `effectiveScopeLabel`:
 - Prefer `propsScopeLabel` from props.
 - Otherwise, compute labels from chosen component/function (e.g. "Component: A > B > C").
- `effectiveScopeFilenameSuffix`:
 - Derived from `effectiveScopeLabel` when no `propsScopeFilenameSuffix` is provided.
 - Re-uses the same sanitization logic (strip prefix before colon, replace spaces, drop special characters).

- **Template and document layout**

- `templates: ExportTemplate[]`
 - Per-project templates, loaded from `localStorage` via `getExportTemplates(projectId)`.
- `selectedTemplateId: string | null`
- `sections: ExportSection[] | undefined`
 - High-level document sections (cover, summary, requirements table, glossary, abbreviations, custom text, placeholder).
- `documentStyle: ExportDocumentStyle | undefined`
 - Styling configuration (header/footer wording, fonts, margins, table colors).
- `useDocumentSections: boolean`
- Template dialogs and state:
 - `isSaveTemplateOpen`, `saveTemplateName`
 - `isManageTemplatesOpen`, `editingTemplateId`, `editingTemplateName`
 - `isCreateTemplateOpen`, `createTemplateFormat`, `createTemplatePreset`, `createTemplateName`
 - `documentPresetSelect`

- **Parameters and glossary**

- `parameterExportMode: ResolveMode ( 'name' | 'resolved' )`
- `includeGlossary: boolean`
- `includeAbbreviations: boolean`
- `glossaryShowDefinitions: boolean`
- `glossarySortAlphabetically: boolean`
- Query-based data:
 - Parameters:
 - Fetched via `parameterService.getParameters(projectId)` (via `useQuery`).
 - Mapped into an efficient lookup (`parameterMap`) keyed by lowercase parameter ID.
 - Definitions:
 - Fetched via `definitionEntryService.getDefinitionEntries(projectId)` (via `useQuery`).
 - Later filtered into glossary vs abbreviation lists based on `type`.
- Parameter placeholders in requirement text are resolved using:
 - `resolveParameterPlaceholders`, working with the `parameterMap` and the chosen `parameterExportMode`.

- **Stepper / UX**

- `currentStep: ExportStep = 'format' | 'scope' | 'options' | 'review'`
 - `inlineError: string | null`
 - Used to surface validation messages inline (e.g., unsupported combinations or missing scope).
 - `scopeResetMessage: string | null`
 - Shows when a loaded template refers to a component/function that no longer exists.
 - `templateUpdatedMessage: string | null`
 - Brief informational message after certain template operations.
 - `isExporting: boolean`
 - Guards against duplicate exports while a file is being generated.
-

Template System Behavior

Templates capture user preferences for repeatable exports and are stored **per project** in local storage.

Template structure

`frontend/src/utils/requirementExportTemplates.ts` defines:

- `ExportTemplateFormat: 'csv' | 'excel' | 'pdf' | 'word' | 'reqif' .`
- `ExportSection :`
 - `id: string` (unique ID).
 - `type :`
`'cover' | 'summary' | 'requirements_table' | 'glossary' | 'abbreviations' | 'custom_text'`
 - `title?: string .`
 - `enabled: boolean .`
 - `options?: { ... } controlling things like:`
 - Cover page options (show project name, date, version, classification, preparer/ organization).
 - Placeholder style (`full_page` vs `heading_with_space`) and how many blank pages to reserve.

- Whether to start a section on a new page; how many blank pages to insert after.
- `ExportDocumentStyle` :
 - Controls:
 - Cover title.
 - Header/footer text.
 - Page number format (`none` | `page` | `pageOfN`).
 - Margins and font sizes.
 - Table header/background/foreground colors.
 - Border colors and alternate row backgrounds.
- `ExportTemplate` :
 - `id` , `name` .
 - `format` .
 - `columns`: `ExportColumn[]` (key/label/selected).
 - `scopeType`: `'all'` | `'component'` | `'function'` .
 - `selectedComponentId?` , `selectedFunctionId?` .
 - `includeHeader` , `parameterExportMode` .
 - `includeGlossary` , `includeAbbreviations` .
 - `glossaryShowDefinitions` , `glossarySortAlphabetically` .
 - Optional `sections` and `documentStyle` .

Storage and lifecycle

- Templates are persisted in `localStorage` under a versioned key per project.
- `getExportTemplates(projectId)` :
 - Reads and parses the stored payload.
 - Returns `[]` on any parse or storage errors.
- `saveExportTemplate(projectId, template)` :
 - Upserts by template `id` and writes back to local storage.
 - Throws a user-friendly `Error` if `QuotaExceededError` is hit.
- `deleteExportTemplate(projectId, templateId)` :
 - Removes the template and rewrites the storage payload.
- `updateExportTemplate(projectId, templateId, patch)` :
 - Applies a partial patch (`Partial<ExportTemplate>`) to an existing template and rewrites storage.
- `mergeColumnsWithDefaults(savedColumns, defaultColumns)` :

- Preserves the `selected` state of columns present in the saved template.
- For default columns not present in the saved template, uses the default configuration.
- Appends any saved columns that don't exist in the defaults (future-proofing).

Applying templates in `ExportBuilder`

When a template is applied:

1. The template's format, columns, header flag, parameter mode, glossary/abbreviation settings, and glossary behaviors are applied to the current export state.
 2. Scope information is interpreted carefully:
 - If the template's `scopeType` references a component or function ID that no longer exists in the current project data:
 - The export scope is reset to `'all'`.
 - `selectedComponentId` / `selectedFunctionId` are cleared.
 - A `scopeResetMessage` is shown to the user, explaining that the saved scope is no longer valid.
 - Otherwise, `scopeType`, `selectedComponentId`, and `selectedFunctionId` are applied directly.
 3. Section and document style settings (if present) are attached:
 - `sections = template.sections`.
 - `documentStyle = template.documentStyle`.
 - `useDocumentSections` is turned on when appropriate.
 4. The wizard step is advanced to the “scope” step to encourage users to confirm or adjust scope after applying a template.
-

Export Formats

The export feature supports five logical formats, all executed client-side:

- **CSV**
- **Excel** (XLSX)
- **PDF**
- **Word** (DOCX)
- **ReqIF**

All formats:

- Operate on `effectiveRequirements` (after any scope and filter logic inside `ExportBuilder`).
- Use the selected `columns` to determine which properties of a requirement are exported.
- Use a computed filename that includes:
 - Project or document title.
 - Timestamp.
 - Optional `scopeFilenameSuffix` .

Below is a conceptual description of each.

CSV / Excel

- Implemented using the `xlsx` library (`XLSX`).
- Data preparation:
 - For each requirement in `effectiveRequirements` :
 - A row object is created where:
 - Keys are column keys (`requirementId` , `title` , `description` , etc.).
 - Values are converted to strings (or left empty when null/undefined).
 - Parameter placeholders may be pre-resolved in text fields depending on `parameterExportMode` .
 - The header row is included or omitted based on `includeHeader` .
- For **Excel**:
 - Rows are assembled into a worksheet.
 - A workbook is created and serialized to a Blob for download.
- For **CSV**:
 - Similar row preparation is done, but the workbook is written out as CSV rather than XLSX.

PDF

- Implemented using:
 - `jsPDF` for document creation.
 - Dynamically imported `jspdf-autotable` for tables (lazy-loaded via `loadAutoTable()` in `ExportBuilder`).
 - Layout helpers in `frontend/src/utils/exportPdfLayout.ts` :

- `getPdfFont`
- `getAuthorityTableStyles`
- `addCoverPage`
- `addHeaderFooterToAllPages`
- `addSectionHeading`
- `addPlaceholderSection`
- Document style:
 - Taken from:
 - The selected `documentStyle` from the template, if any.
 - Otherwise, `DEFAULT_AUTHORITY_STYLE` .
 - Controls:
 - Font family mapping (via `getPdfFont`).
 - Margins.
 - Header/footer text and page numbering style.
 - Table header background/foreground colors.
 - Alternate row background color.
 - Border color and line width.
- Sections:
 - When `sections` are defined:
 - A cover page can be added via `addCoverPage` , using the document title, project name, date, version, classification, and preparer/organization from the section's `options` .
 - Additional sections (summary, requirements table, glossary, abbreviations, placeholders) are rendered in order:
 - Summary:
 - Shows a heading and brief count of requirements.
 - Optionally includes project name.
 - Requirements table:
 - Uses `jspdf-autotable` with styles from `getAuthorityTableStyles` .
 - Uses the selected columns as table headers.
 - Glossary and abbreviations:
 - Built from mapped definition entries (see below).
 - Placeholder sections:
 - Created using `addPlaceholderSection` , reserving blank pages or adding heading-only placeholders depending on options.

- When no sections are defined:
 - A simpler default table or layout is used (implementation-specific) but still respects the chosen columns and style where possible.
- After all content is added:
 - `addHeaderFooterToAllPages` is called to overlay headers and footers on every page using the style's placeholders (`{page}` , `{pageOfN}` , `{date}` , `{title}`).

Word (DOCX)

- Implemented using:
 - The `docx` library (dynamically imported inside `buildRequirementsDocxWithSections`).
 - Helpers in `frontend/src/utils/exportDocx.ts` :
 - `buildRequirementsDocxWithSections`
 - `DocxRequirementRow` , `DocxColumn` , `DocxGlossaryEntry` .
- Data preparation:
 - Requirements are mapped into `DocxRequirementRow[]` where keys match the selected column keys.
 - Glossary and abbreviation entries are mapped into:
 - `DocxGlossaryEntry[]` for each type, using definition entries fetched from `definitionEntryService` .
 - A `BuildDocxWithSectionsOptions` object is formed containing:
 - `documentTitle`
 - `projectName`
 - `requirements`
 - `columns`
 - `sections`
 - `documentStyle`
 - `glossaryEntries`
 - `abbreviationEntries`
 - An optional `stripHtml` function to clean rich text fields.
- Layout behavior:
 - Uses the `docx` constructs (`Document` , `Paragraph` , `Table` , etc.) to create:
 - Cover page with title, project name, date, version, classification, and preparer/organization.
 - Summary section with requirement count and optional project name.
 - Requirements table section with:

- A heading (e.g. `1. Requirements`).
- A table where:
 - Header row labels come from the selected columns.
 - Cells are created from requirement row values, with HTML stripped.
- Glossary and abbreviations sections with two-column tables (“Term” / “Definition”).
- Placeholder sections for future manual content.
- Styles such as:
 - Font family.
 - Heading vs body font sizes.
 - Table borders and header shading.
- Are derived from `documentStyle` or `DEFAULT_AUTHORITY_STYLE` .

ReqIF

- The ReqIF export format targets interoperability with requirements tools that support the ReqIF standard.
 - Conceptually:
 - Requirements are mapped into ReqIF “specification objects”.
 - Selected columns become ReqIF attributes.
 - The implementation builds an XML/ReqIF document that includes:
 - The set of exported requirements (from `effectiveRequirements`).
 - Required metadata such as specification IDs, attribute definitions, and values.
 - The actual schema-specific details are encapsulated inside `ExportBuilder` ; from the outside, ReqIF behaves like another format choice:
 - It appears in the format selector.
 - When selected and confirmed, it produces a downloadable ReqIF file.
-

Parameters & Glossary Integration

Parameters

- Fetched via `parameterService.getParameters(projectId)` using `@tanstack/react-query` .
- Mapped into a `Map<string, { id; name; defaultValue; unit; tolerance }>` keyed by **lowercase parameter ID** for quick lookup.

- Used in conjunction with `resolveParameterPlaceholders` to process requirement text:
 - Parameter placeholders embedded in requirement titles/descriptions (or other fields) can be replaced with:
 - The parameter **name** (when `parameterExportMode === 'name'`).
 - The **resolved value** (value + unit/tolerance) when `parameterExportMode === 'resolved'` .
- This means exported documents can:
 - Retain symbolic placeholders.
 - Or be fully “concrete” with numeric values, depending on user choice.

Definitions (glossary and abbreviations)

- Fetched via `definitionEntryService.getDefinitionEntries(projectId)` using `@tanstack/react-query` .
 - Filtered into:
 - Glossary entries (`type === 'glossary'`).
 - Abbreviation entries (`type === 'abbreviation'`).
 - Each entry is mapped into `DocxGlossaryEntry` for Word and similar structures for PDF tables.
 - Behavior controlled by:
 - `includeGlossary` / `includeAbbreviations` flags.
 - `glossaryShowDefinitions` :
 - Controls whether full definitions are included or only terms.
 - `glossarySortAlphabetically` :
 - Determines whether entries are sorted by term before export.
-

UX, Validation & Keyboard Behavior

- **Escape key:**
 - Listened for at window level while the modal is open.
 - Closes the modal via `onClose()` (which in turn flips `isExportOpen` off in `RequirementsPage`).
- **Step navigation:**
 - The UI is organized into discrete steps (`currentStep`), typically:

- Format selection.
 - Scope selection.
 - Options (columns, parameters, glossary, templates).
 - Review and export.
 - Buttons allow moving forward/back; navigation is guarded by lightweight validation where needed.
 - **Inline errors:**
 - `inlineError` is used to show problems like:
 - Empty template names.
 - Invalid scope selections.
 - Other configuration issues that prevent export.
 - **Disabled actions:**
 - Buttons like “Export” are disabled when `isExporting` is true or when minimal required information is not available.
 - In baseline contexts, import/export controls in `RequirementsPage` are conditionally disabled where appropriate (e.g., import is disabled for baselines).
-

Edge Cases, Limitations & Considerations

- **Large datasets:**
 - `allRequirements` is cached for 30 seconds via React Query (`staleTime: 30_000`) to reduce refetching and improve performance on repeated exports.
 - Very large exports depend on the browser’s memory and the performance characteristics of `XLSX` , `jsPDF` , and `docx` .
- **LocalStorage limits:**
 - Template operations can fail if `localStorage` is full; in this case, a user-facing error is thrown recommending deleting old templates or freeing browser storage.
- **Dynamic imports:**
 - `docx` and `jspdf-autotable` are loaded dynamically:
 - This helps avoid build-time issues and reduces initial bundle size.
 - If these imports fail (e.g., due to environment issues), the corresponding export formats (PDF/Word) will not work and an error is logged/surfaced.
- **Scope template drift:**
 - Saved templates may refer to components or functions that no longer exist in the

current project data.

- In such cases, scope is reset to “All” with a `scopeResetMessage` , but the rest of the template (format, columns, glossary options) remains usable.
- **Baseline-specific behavior:**
 - When in a baseline view, certain data-management actions such as import may be disabled to preserve baseline immutability.
 - Export still operates on the requirements available in the current baseline/context, and `RequirementsPage` is responsible for passing the correct set.

This document reflects the current behavior of the Requirements Export functionality as implemented in `RequirementsPage` and `ExportBuilder` , plus their associated utilities and services.